

А. Анализ счетов

1 секунда, 256 мегабайт

Алексей — сотрудник передовой IT-компании, недавно он начал работать в отделе, который занимается исследованием теории чисел. Алексей работает в банке, и недавно их отдел выяснил, что поведение клиента зависит от делителей количества рублей на его счете. Особенно важны большие делители.

Обозначим количество рублей на счёте клиента числом n . Для улучшения взаимодействия с клиентами банку необходимо узнать, сколько делителей больше или равных 10^{15} у числа n .

Помогите Алексею справиться с этой задачей.

Входные данные

Первая строка содержит одно целое число t ($1 \leq t \leq 1000$) — количество наборов входных данных. Далее следует описание наборов входных данных.

Каждый набор входных данных содержит в единственной строке одно целое число n — количество рублей на счете клиента ($1 \leq n \leq 10^{18}$).

Выходные данные

Для каждого набора входных данных выведите одно целое число — количество делителей числа n , которые больше или равны 10^{15} .

входные данные
3
10
1000000000000000
20000000000000001
выходные данные
0
1
2

В. Пицца на Мышечных технологиях

2 секунды, 256 мегабайт

На одной из площадок олимпиады по физкультуре «Мышечные технологии» есть две аудитории. Между ними расположены n столов, на которых лежат пиццы, на i -м столе находятся a_i пицц. При этом *первая* аудитория находится перед первым столом, а *вторая* — после последнего стола.

По завершении олимпиады, каждую секунду, начиная с первой, ровно один участник выходит из **каждой** аудитории, находит ближайший к нему стол, на котором еще лежит **хотя бы одна** пицца, подходит к нему и ест ровно одну пиццу, после чего сразу же уходит на разбор. Участники очень старались на олимпиаде, поэтому проголодались. Можно считать, что они моментально подходят к столу и едят пиццу.

В разных аудиториях участникам были предложены разные задачи. Поэтому, если участник из *первой* аудитории видит, как участник из *второй* аудитории ест пиццу со стола на расстоянии **не больше** k , он немедленно начнёт *обсуждать с ним задачи*. Или же, более формально, пусть для участника из *первой* аудитории x — номер ближайшего к нему стола с пиццей, а y — номер стола с пиццей для участника из *второй* аудитории, тогда, если $y - x \leq k$, то они немедленно начнут *обсуждать задачи*.

Организаторы устали после тура и хотят отдохнуть в тишине. Им известно, что как только участники начнут *обсуждать задачи*, их покой будет нарушен. Помогите организаторам определить, через сколько секунд после завершения олимпиады начнётся *обсуждение задач*.

Входные данные

Данная задача состоит из нескольких наборов входных данных. В первой строке задано единственное число t ($1 \leq t \leq 10^4$) — количество наборов входных данных. Затем идет их описание.

В первой строке каждого набора входных данных заданы два числа n, k ($2 \leq n \leq 2 \cdot 10^5, 1 \leq k \leq n - 1$) — количество столов с пиццами между аудиториями и расстояние, на котором участники начнут *обсуждать задачи*.

В следующей строке заданы целые числа $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — количества пицц на столах.

Гарантируется, что сумма n по всем наборам входных данных не превосходит $2 \cdot 10^5$.

Выходные данные

Для каждого набора входных данных выведите ровно одно число — минимальное количество секунд, после которых участники начнут *обсуждать задачи*.

Можно показать, что в указанных ограничениях *обсуждение задач* гарантированно произойдёт.

входные данные
3
5 2
1 1 1 1 2
5 3
1 1 1 1 2
3 1
3 1 4
выходные данные
3
2
4

В первых двух наборах входных данных массив количеств пицц имеет вид [1, 1, 1, 1, 2].

На первой секунде участник из *первой* аудитории взял пиццу под номером 1, а из *второй* 5, после этого массив количеств пицц имеет вид [0, 1, 1, 1, 1].

На второй секунде участник из *первой* аудитории взял пиццу под номером 2, а из *второй* 5, после этого массив количеств пицц имеет вид [0, 0, 1, 1, 0].

На третьей секунде участник из *первой* аудитории взял пиццу под номером 3, а из *второй* 4, после этого массив количеств пицц имеет вид [0, 0, 0, 0, 0].

В первом наборе входных данных $k = 2$, поэтому на третьей секунде они начнут *обсуждать задачи*, так как $4 - 3 \leq k$.

Во втором наборе входных данных они начнут *обсуждать задачи* на второй секунде.

В третьем наборе входных данных они начнут *обсуждать задачи* на четвёртой секунде.

С. Телепортация в Немалополисе

2 секунды, 512 мегабайт

Недавно в Немалополисе разработали инновационный метод телепортации. Для повышения комфорта жителей в городе было установлено n телепортов, пронумерованных от 1 до n .

Учёные-немалисты рассчитали ограничения, которые обеспечат безопасность телепортационной сети. Для каждого телепорта было выписано слово s_i соответственно. За $|s_i|$ они обозначили длину строки i . Также они выбрали целое число k . Учёные разрешили горожанам телепортироваться из телепорта i в телепорт j при условии, что пассажир предъявит такое число m , что:

- суффикс s_i длины m равен префиксу s_j длины m ;
- суффикс s_i длины $m + 1$ **не** равен префиксу s_j длины $m + 1$ (в частности, если $m + 1 > \min$);
- $k \cdot m \leq \min(|s_i|, |s_j|)$ (по Федеральному Закону «О немалости операций»).

При этом стоимость такой телепортации составляет m немалей (местная валюта).

Например, для $s_i = \text{«abcde»}$ и $s_j = \text{«cdef»}$ подходит только $m = 3$, и стоит такая телепортация будет 3 немалей.

Мэр города, Немалуй, собирает сведения об эффективности телепортов. Для каждого телепорта i ($1 \leq i \leq n$) вам нужно найти минимальное число немалей, необходимое для того, чтобы добраться от 1-го телепорта до телепорта i , совершив какое-то количество телепортаций.

Входные данные

Каждый тест состоит из нескольких наборов входных данных. Первая строка содержит одно целое число t ($1 \leq t \leq 2 \cdot 10^4$) — количество наборов входных данных. Далее идут описания наборов входных данных.

Первая строка каждого набора входных данных содержит два целых числа n ($2 \leq n \leq 3 \cdot 10^5$) и k ($1 \leq k \leq 3 \cdot 10^5$) — количество телепортов и ограничение на стоимость телепортации соответственно.

Следующие n строк каждого набора входных данных содержат s_i ($1 \leq |s_i| \leq 3 \cdot 10^5$), состоящие только из строчных латинских букв — надписи над соответствующими телепортами.

Гарантируется, что сумма $|s_i|$ по всем наборам входных данных не превышает $3 \cdot 10^5$.

Выходные данные

Для каждого набора входных данных в отдельной строке выведите $n - 1$ число — стоимости проезда от телепорта 1 до телепортов 2, 3, 4, ..., n соответственно. Если до какого-то телепорта никак нельзя доехать, выведите вместо стоимости проезда до него число -1.

входные данные
5 5 1 skolko nemalo kolba zamkadom aramzamzam 4 2 arbuzik ikra pupupu kokos 2 1 uuuuu uuuuu 2 1 uuuuu uuuuu 4 7 extraordinarilmagnificentunbelievabilityoverenthusiasticalllyhyperactiveoverenthusiasticallyhyperactiveperpetualmotionunintentionallymisunderstoodhilariousscenariofantasticallyextraordinari fantasticallyextraordinarilybizarreadventure
выходные данные
-1 2 6 3 2 -1 -1 3 5 31 59 85

В первом наборе входных данных мы можем телепортироваться: «skolko»^{{2}{to}} «kolba»^{{1}{to}} «aramzamzam»^{{3}{to}} «zamkadom».

Во втором наборе входных данных мы не можем попасть из «arbuzik» в «kokos», так как проезд между ними стоит 1, что меньше $k = 2$.

В третьем наборе входных данных мы не можем попасть из «uuuuu» в «uuuuu», предъявив $m = 1$ или $m = 2$, так как условие на то, что суффикс s_i длины $m + 1$ не совпадает с префиксом s_j длины $m + 1$, не выполнено.

В четвёртом наборе входных данных мы можем попасть из «uuuuu» в «uuuuu» только при $m = 5$, так как при меньших условие на $m + 1$ не выполнено.

Пятый набор входных данных настолько немалый, что его мы комментировать не будем.

D. Перетировка сорстановки

2 секунды, 256 мегабайт

Вам дана перестановка p всех чисел от 1 до n , а также целое число k . С перестановкой разрешается выполнять следующую операцию неограниченное число раз:

- Выбрать любой подотрезок длины k .
- Найти на этом подотрезке минимальный и максимальный элементы.
- Поменять их местами.

Ваша задача с помощью этих операций отсортировать перестановку или же сообщить, что это невозможно.

Входные данные

Первая строка содержит одно целое число t ($1 \leq t \leq 100$) — количество наборов входных данных. Далее следует описание наборов входных данных.

Первая строка каждого набора входных данных содержит два целых числа n, k ($5 \leq n \leq 100, 2 \leq k \leq 4$) — длина перестановки, а также размер отрезка, над которым проводим операцию.

Вторая строка каждого набора входных данных содержит числа $p_1, p_2, \ldots, p_n$ ($1 \leq p_i \leq n$) — изначальная перестановка.

Гарантируется, что сумма n по всем наборам входных данных не превышает 100, а также, что p является корректной перестановкой.

Выходные данные

Для каждого набора входных данных выведите «YES», если возможно отсортировать перестановку с помощью таких операций, в ином случае «NO».

Если можно отсортировать, то затем нужно вывести сами операции.

В первой строке выведите число m — количество действий для сортировки. В следующих m ($1 \leq m \leq 10^5$) строках сами операции в порядке выполнения.

Для каждой операции выведите два числа l, r ($1 \leq l \leq r \leq n, r - l + 1 = k$) — означающие отрезки, над которыми проводятся операции.

Гарантируется, что для всех входных данных, где ответ существует, существует набор операций, состоящий не более чем из 10^5 операций, который позволяет отсортировать массив.

входные данные
4 8 2 1 4 8 3 2 5 7 6 5 3 1 3 5 2 4 5 4 1 3 2 5 4 6 3 4 3 5 1 6 2
выходные данные
YES 9 3 4 2 3 4 5 3 4 2 3 5 6 6 7 7 8 6 7 YES 7 1 3 3 5 1 3 2 4 3 5 2 4 1 3 NO YES 6 1 3 2 4 1 3 4 6 3 5 2 4

E. Warfare 1917: Remastered

2 секунды, 256 мегабайт

Это интерактивная задача с открытыми тестами.

Наконец-то выпустили долгожданный ремастер вашей любимой *Flash* игры *Warfare 1917*! Вы так долго его ждали, что отложили все свои дела и бросились играть. Работавшие над игрой креативные менеджеры решили, что предпочтения игроков за последние 17 лет сильно поменялись, поэтому многие игровые механики были упрощены. В то же время менеджеры постарались сохранить ~~бюджет~~ дух оригинала, поэтому приняли решение переиспользовать старые графические ресурсы.

В игре сражаются два игрока, каждый имеет на руках карты со значениями от 1 до 6, соответствующие юнитам из оригинальной игры, и по **ноль** очков. Карта с **большим** значением бьёт карту с **меньшим**. Игра проходит в шесть *раундов*, каждый *раунд* игроки одновременно выбирают и разыгрывают по карте. Если значения карт совпадают, ничего не происходит, в противном случае игрок с большим значением получает одно *очко*. Разыгранные карты переиспользовать нельзя. Если один из игроков набрал больше *очков*, чем другой, он объявляется победителем игры.

Вы будете сражаться с ботами, которые играют по следующим стратегиям:

- 1. Каждая карта — минимальная в руке бота (в данный момент);
- 2. Каждая карта — максимальная в руке бота (в данный момент);
- 3. Первые три карты — 1, 2, 3 (в случайном порядке), следующие три карты — 4, 5, 6 (в случайном порядке);
- 4. Первые три карты — 4, 5, 6 (в случайном порядке), следующие три карты — 1, 2, 3 (в случайном порядке);
- 5. Каждая карта — случайная из **двух** минимальных в руке бота (в данный момент);
- 6. Каждая карта — случайная из **двух** максимальных в руке бота (в данный момент);
- 7. Каждая карта — случайная из **трёх** минимальных в руке бота (в данный момент);
- 8. Каждая карта — случайная из **трёх** максимальных в руке бота (в данный момент).

Каждый тест соответствует **одному** боту, при этом порядок тестов соответствует порядку ботов в описании. В каждом тесте вы сыграете с ботом 500 независимых игр. Ваша задача — выиграть у **каждого** бота хотя бы 251 игру. Ничья не считается за победу.

Обратите внимание, что боты **не меняют** свою стратегию, но при этом *случайность* равномерно распределена и работает независимо для каждой игры. Например, третий бот может сыграть последовательность карт 1, 3, 2, 5, 6, 4 в первой игре и 2, 3, 1, 4, 5, 6 во второй.

Протокол взаимодействия

Вы сыграете с каждым ботом 500 раз.

Каждая игра проходит в 6 *раундов*. Каждый *раунд* вы должны сначала вывести значение карты, которой хотите сходить, а затем считать значение карты бота.

После окончания игры начинается следующая, и вы играете её в таком же формате, пока не сыграете 500 игр. Вы должны доигрывать все *раунды* и все игры, нельзя завершить программу, если вы уже набрали достаточное количество очков.

Если ваша программа нарушит протокол взаимодействия, сделает некорректный ход или зависнет, вы можете получить случайный вердикт ошибки.

Не забывайте сбрасывать буфер после **каждого** вывода. Для этого можете использовать `std::endl` в C++ или стандартную функцию `print` в Python. Не используйте `ios_base::sync_with_stdio(false)` в C++.

Интерактор **не является** адаптивным, то есть ходы ботов зафиксированы заранее и **не зависят** от ваших действий.

входные данные
1
2
3
4
5
6
...
выходные данные
4
2
1
5
6
3
...

В примере показано взаимодействие программы участника с первым ботом. В нём сыграна одна игра, в которой участник набрал 3 очка, а бот — 2 очка. Эту игру выиграл участник, далее ему предстоит сыграть ещё 499 игр с этим же ботом.

F. Артем и Массив

2 секунды, 256 мегабайт

У Артема есть массив a длины n . Ему не нравится, когда в массиве есть различные значения, поэтому он просит вас исправить это.

Вам разрешено выполнять операции, которые преобразуют массив. За одну операцию вы можете выбрать три целых числа l, r, x , если $\lceil \frac{n}{2} \rceil \leq r - l + 1$. Тогда для всех i , что $l \leq i \leq r$, применяется изменение $a_i = a_i \oplus x$, где $\oplus$ обозначает побитовое «исключающее или».

Скажите, можно ли сделать все элементы массива равными, выполнив не более $n - 1$ операций.

Входные данные

Каждый тест состоит из нескольких наборов входных данных. Первая строка содержит одно целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных. Затем следует описание наборов входных данных.

Первая строка каждого набора входных данных содержит единственное число n ($1 \leq n \leq 2 \cdot 10^5$) — длина массива.

Вторая строка каждого набора входных данных содержит n целых чисел $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 2^{30}$) — сам массив a .

Гарантируется, что сумма значений n по всем наборам входных данных не превосходит $2 \cdot 10^5$.

Выходные данные

Для каждого набора входных данных в отдельной строке выведите одно целое число — количество операций k ($0 \leq k \leq n - 1$), или -1 , если сделать числа равными невозможно.

В случае, если ответ существует, в следующих k строках выведите по три целых числа l, r, x ($1 \leq l \leq r \leq n, 0 \leq x \leq 2^{30}$) — описание операций.

входные данные
3
4
179 179 179 179
9
4 5 5 7 7 7 6 6 4
7
43 23 483 34 2 48 29
выходные данные
0
2
2 6 1
4 8 2
6
1 5 50
2 7 60
3 7 500
1 6 45
4 7 449
1 4 32

В первом наборе входных данных все числа массива равны, поэтому нам не надо выполнять операции.

Во втором наборе данных достаточно двух операций:

- 1. Выполним операцию со значениями $l = 2, r = 6, x = 1$. После её применения изменятся все элементы со 2-го по 6-й включительно. Тогда массив будет иметь вид $[4, 4, 4, 6, 6, 6, 6, 4]$.
- 2. Выполним операцию со значениями $l = 4, r = 8, x = 2$. После её применения изменятся все элементы с 4-го по 8-й включительно. Тогда массив будет иметь вид $[4, 4, 4, 4, 4, 4, 4, 4]$.

G. Генератор деревьев

3 секунды, 512 мегабайт

Назовём массив $b_1, b_2, \dots, b_{2k-1}$, k -древесным, для целого $k \geq 2$, если **его элементы можно переставить** таким образом, чтобы одновременно были выполнены следующие условия:

- $b_1 = k$
- Если рассмотреть граф, в котором есть вершины, соответствующие всем различным значениям среди $b_2, b_3, \dots, b_{2k-1}$, и провести в нём $k-1$ рёбер между вершинами с значениями b_2 и b_3, b_4 и $b_5, \dots, b_{2k-2}$ и b_{2k-1} , то получившийся граф должен быть деревом

Более неформально, после перестановки массив должен представлять собой то, как чаще всего задаётся в входных данных дерево размера k : первым элементом является k , количество вершин, а далее задаётся $k-1$ рёбер, и каждое ребро задаётся двумя вершинами. Обратите внимание, что номерами вершин могут быть любые целые числа.

Например, массив $[4, 100, 4, 3, 3]$ является 3-древесным, так как его элементы можно переставить в $[3, 3, 4, 4, 100]$, и граф с рёбрами $3 \rightarrow 4, 4 \rightarrow 100$ является деревом из 3 вершин.

Вам дан массив $a_1, a_2, \dots, a_n$ и q запросов. В каждом запросе даны два числа $l \leq r$, и нужно найти количество целых $k \geq 2$ таких, что у массива $a_l, \dots, a_r$ есть хотя бы одна k -древесная подпоследовательность.

Напомним, что граф на k вершинах является деревом, если он является связным, то есть между каждой парой вершин существует путь по рёбрам графа, и содержит ровно $k-1$ рёбер. В частности, дерево не содержит петель и кратных рёбер.

Входные данные

Каждый тест состоит из нескольких наборов входных данных. В первой строке находится одно целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных. Далее следует описание наборов входных данных.

Первая строка каждого набора входных данных содержит два целых числа n, q ($1 \leq n \leq 10^5, 1 \leq q \leq 10^5$) — длина массива и количество запросов.

Вторая строка каждого набора входных данных содержит n целых чисел $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 2n$) — массив a .

Каждая из последующих q строк набора входных данных содержит два целых числа l, r ($1 \leq l \leq r \leq n$) — параметры запроса.

Гарантируется, что сумма n по всем наборам входных данных не превосходит 10^5 , а также сумма q по всем наборам входных данных не превосходит 10^5 .

Выходные данные
Выведите ответ на каждый запрос в отдельной строке.

входные данные
1
10 15
6 1 13 6 4 4 1 5 2 5
1 10
1 9
2 10
1 8
2 9
3 10
1 7
2 8
3 9
4 10
1 6
2 7
3 8
4 9
5 10
выходные данные
3
3
2
1
1
1
1
0
1
1
0
0
0
1
1

В первом тестовом примере, в первом запросе существуют следующие древесные подпоследовательности:

- 2-древесная подпоследовательность [6, 4, 2]. Элементы этого массива можно переставить в [2, 4, 6] и получить дерево.
- 4-древесная подпоследовательность [1, 4, 4, 1, 5, 2, 5]. Элементы этого массива можно переставить в [4, 4, 5, 5, 1, 1, 2] и получить дерево.
- 5-древесная подпоследовательность [6, 1, 13, 6, 4, 4, 1, 2, 5]. Элементы этого массива можно переставить в [5, 2, 1, 1, 6, 6, 4, 4, 13] и получить дерево.

Эти три дерева представлены на картинках ниже.

Н. Игра с заменой карт

1 секунда, 256 мегабайт

Processing math: 13%

Это интерактивная задача!

Алиса и Боб играют в следующую игру. Изначально у Алисы есть n карточек с числами $a_1, a_2, \dots, a_n$ из неотрицательных чисел, меньших m . Также задана фиксированная бинарная строка $s_0s_1\dots s_{m-1}$.

Каждый ход Алиса может совершить одно из следующих действий:

- Выбрать две свои карточки. Пусть числа, записанные на них — x и y . Тогда Алиса убирает обе карточки с числами x, y и заменяет их одной карточкой с числом $(x + y) \bmod m$.
- Выбрать одну свою карточку. Пусть число, записанное на ней — x . Тогда Алиса может попросить Боба заменить эту карточку на некоторое число от 0 до $m-1$ включительно **не равное** x . Обратите внимание, что число на новой карточке выбирает Боб.
- Закончить игру.

Если через $n + m + 7$ ходов Алиса не закончила игру, или у неё на руках всё ещё больше одной карты, победителем считается Боб. Иначе у Алисы на руках одна карточка. Пусть на этой карточке число x . Тогда, если $s_x = 1$, в игре побеждает Алиса, а иначе побеждает Боб.

Ваша задача — выбрать, за какого игрока играть, и выиграть игру с программой жюри.

Входные данные
Каждый тест состоит из нескольких наборов входных данных. В первой строке находится одно целое число t ($1 \leq t \leq 100$) — количество наборов входных данных. Далее следует описание наборов входных данных.

Протокол взаимодействия
Взаимодействие начинается со считывания входных данных. Первая строка каждого набора входных данных содержит два целых числа n, m ($2 \leq n \leq 100, 2 \leq m \leq 100$) — длину массива a и модуль.

Вторая строка каждого набора входных данных содержит n целых чисел $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq m$) — элементы массива a .

Третья строка каждого набора входных данных содержит бинарную строку длины m — $s_0s_1\dots s_{m-1}$.

После считывания данных вы должны вывести одну строку: `Alice` или `Bob` — имя игрока, за которого вы будете играть.

Если вы выбрали играть за Алису, далее выводите совершаемые операции в следующем формате:

- `unite x y` — для объединения карточек с числами x и y в одну карточку с числом $(x + y) \bmod m$.
- `replace x` — для замены карточки с значением x . После вывода в одной строке считайте число y ($0 \leq y \leq m, y \neq x$) — значение новой карточки, выбранное Бобом.
- `done` — закончить игру.

Если вы выбрали играть за Боба, далее считывайте операции Алисы в таком же формате. На каждую операцию `replace x` — вам нужно вывести число y ($0 \leq y \leq m, y \neq x$) в отдельной строке. Также гарантируется, что программа жюри, играющая за Алису, закончит игру за не более чем $n + m + 7$ ходов, с одной картой на руках.

После вывода или считывания `done` вы должны перейти к следующему набору входных данных.

После вывода каждого запроса не забудьте вывести перевод строки и сбросить буфер вывода^{^{\text{*}}}. В противном случае вы получите вердикт `Решение «зависло»`. На любом шаге взаимодействия, если вы считали `-1` вместо корректных данных, ваше решение должно немедленно завершиться. Это означает, что ваше решение получит вердикт `Неправильный ответ` из-за некорректного запроса или любой другой ошибки. Если программа не завершится, вы можете получить любой вердикт, так как ваша программа продолжит чтение из закрытого потока.

^{^{\text{*}}}Чтобы сбросить буфер вывода, используйте:

- `fflush(stdout)` или `cout.flush()` в C++;
- `sys.stdout.flush()` в Python;
- смотрите документацию для других языков.

ВХОДНЫЕ ДАННЫЕ
<pre>2 5 4 0 2 0 3 3 0110 1 0 2 10 7 7 111101111 unite 7 7 done</pre>
ВЫХОДНЫЕ ДАННЫЕ
<pre>Alice replace 0 unite 3 3 unite 1 2 replace 3 unite 0 0 unite 0 2 done Bob</pre>

Не гарантируется, что в примере из условия жюри играет оптимально.

I. Объединение деревьев

2 секунды, 256 мегабайт

У вас есть два корневых дерева t_1 и t_2 , состоящие из n_1 и n_2 вершин соответственно.

Корневое дерево — это дерево с выделенной вершиной, которую называют *корнем*.

В каждой вершине деревьев t_1 и t_2 записано целое число — *вес* данной вершины.

Весом дерева называется целое число, которое получается при последовательном выполнении следующих двух операций:

Processing math: 13%

1. Спуститься по дереву от корня до каждого листа, считая XOR весов всех вершин на пути.
2. Сложить получившиеся значения для всех листов.

XOR — это операция побитового исключающего ИЛИ, обозначаемая символом `\oplus`. При вычислении выражения $a \oplus b$ каждый бит результата будет равен 0, если соответствующие биты чисел a и b равны, и равен 1 иначе.

Например, вес дерева на рисунке равен 16. Он может быть рассчитан следующим образом:

1. XOR весов на пути до первого листа равен $3 \oplus 6 = 5$.
2. XOR весов на пути до второго листа равен $3 \oplus 4 = 7$.
3. XOR весов на пути до третьего листа равен $3 \oplus 2 \oplus 1 = 0$.
4. XOR весов на пути до четвёртого листа равен $3 \oplus 2 \oplus 5 = 4$.
5. Вес всего дерева равен $5 + 7 + 0 + 4 = 16$.

Корневое дерево из 6 вершин.

Вам необходимо подвесить одно из деревьев за другое таким образом, чтобы вес полученного дерева получился **максимальным**.

Другими словами, необходимо выполнить одну из следующих операций:

- Выбрать вершину i ($1 \leq i \leq n_1$) в дереве t_1 и провести от неё ребро к вершине, являющейся корнем дерева t_2 .
- Выбрать вершину j ($1 \leq j \leq n_2$) в дереве t_2 и провести от неё ребро к вершине, являющейся корнем дерева t_1 .

В результате получится новое дерево t , состоящее из $n_1 + n_2$ вершин. Необходимо найти дерево t такое, вес которого будет **максимальным** среди всех возможных деревьев, которые можно получить выполнением одной из операций выше.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

Первая строка каждого набора входных данных содержит ровно одно целое число n_1 ($1 \leq n_1 \leq 10^5$) — количество вершин в дереве t_1 .

Вторая строка каждого набора входных данных содержит ровно n_1 целых чисел — элементы массива w_1 , где $w_{\{1_i\}}$ ($1 \leq w_{\{1_i\}} \leq 10^9$) описывает вес в вершине дерева t_1 с номером i ($1 \leq i \leq n_1$).

Третья строка каждого набора входных данных содержит ровно n_1 целых чисел — элементы массива p_1 предков дерева t_1 . Элемент $p_{\{1_i\}}$ содержит номер родителя вершины i . Если $p_{\{1_i\}} = i$, то вершина i является корнем дерева t_1 .

Четвёртая строка каждого набора входных данных содержит ровно одно целое число n_2 ($1 \leq n_2 \leq 10^5$) — количество вершин в дереве t_2 .

Пятая строка каждого набора входных данных содержит ровно n_2 целых чисел — элементы массива w_2 , где $w_{\{2_i\}}$ ($1 \leq w_{\{2_i\}} \leq 10^9$) описывает вес в вершине дерева t_2 с номером i ($1 \leq i \leq n_2$).

Шестая строка каждого набора входных данных содержит ровно n_2 целых чисел — элементы массива p_2 предков дерева t_2 . Элемент $p_{\{2_i\}}$ содержит номер родителя вершины i . Если $p_{\{2_i\}} = i$, то вершина i является корнем дерева t_2 .

Гарантируется, что сумма n_1 и n_2 по всем наборам входных данных не превышает $2 \cdot 10^5$.

Выходные данные

Для каждого набора входных данных на отдельной строке выведите ровно одно число — максимальный вес, который можно получить, если подвесить одно дерево за другое.

входные данные
3
6
3 6 4 2 1 5
1 1 1 1 4 4
3
1 1 1
1 1 1
2
100 101
1 1
2
1 1
2 2
4
6 6 6 1
4 3 4 4
4
10000 1 1 10000
2 3 3 2
выходные данные
23
101
30008

В первом наборе входных данных

- Вес дерева t_1 равен 16 — оно соответствует дереву, приведённому в тексте условия.
- Вес дерева t_2 равен 0. Оно состоит из 3-х узлов, два из которых являются листьями, поэтому вес вычисляется как $(1 \oplus 1) + (1 \oplus 1) = 0 + 0 = 0$.

Дерево t максимального веса может быть получено, если подвесить дерево t_2 к вершине номер 3 (имеющей вес 4) дерева t_1 , как показано на рисунке ниже:

Вес данного дерева равен 23.

J. Позитивная игра

4 секунды, 256 мегабайт

Мадока не смогла выиграть олимпиаду *Когнитивные технологии*, поэтому попытается выиграть хотя бы своего лучшего друга. Они будут играть в следующую игру.

У Мадоки есть связный взвешенный неориентированный граф, состоящий из n вершин, пронумерованных числами от 1 до n . Она выбирает в нём подмножество из n рёбер, таких, что подграф, образованный этими рёбрами и вершинами на их концах, является связным. Затем второй игрок выбирает из этого подмножества ребро, после удаления которого подграф останется связным, и удаляет его.

Счётом игры называется суммарный вес оставшихся $n - 1$ рёбер в этом подмножестве. Первый игрок (Мадока) хочет **минимизировать** счёт, тогда как второй игрок (лучший друг Мадоки) хочет его **максимизировать**.

Мадока хочет узнать счёт игры, при условии, что оба игрока играют оптимально. Помогите ей с этим.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 1000$) — количество наборов входных данных.

Далее следует описание наборов.

В первой строке даны целые числа n , m ($3 \leq n \leq 15$, $n \leq m \leq \frac{n(n-1)}{2}$) — количество вершин в графе и количество рёбер.

В следующих m строках дано описание рёбер.

Каждое ребро задаётся целыми числами a , b , c ($1 \leq a, b \leq n$, $a \neq b$, $1 \leq c \leq 10^9$), обозначающие ребро между вершинами a и b с весом c .

Гарантируется, что граф не содержит кратных рёбер, а также является связным.

Также гарантируется, что **не более** чем в одном наборе входных данных $n \geq 8$.

Выходные данные

Для каждого набора входных данных выведите единственное число — *счёт* игры, если оба игрока играют оптимально.

входные данные
1
4 6
2 3 6
2 1 4
3 1 7
4 3 7
1 4 8
2 4 2
выходные данные
15

К. Ксюша против гориллы

2 секунды, 256 мегабайт

Ксюша — именитый эксперт в области компьютерной безопасности. Известная корпорация, разрабатывающая программное обеспечение RISC-V процессоров, наняла Ксюшу для расследования недавнего инцидента. Неопознанный горилл пробрался в продовый контур корпорации и похитил секретную разработку — массив из n **положительных** целых чисел $a_1, a_2, \ldots, a_n$. Известно, что этот массив был упорядочен по **неубыванию** ($1 \leq a_1 \leq a_2 \leq \ldots \leq a_n$).

Неопознанный горилл — опытный хакер, поэтому он придумал, как обойти периметр фильтрации внутреннего трафика и извлечь массив a во внешнюю сеть. Горилл дописал в конец массива a элементы $a_1, a_1 + a_2, \ldots, a_1 + a_2 + \ldots + a_n$, другими словами, элементы массива префиксных сумм a . Затем горилл перемешал получившийся массив длины $2n$ случайным образом и назвал его b .

Ксюша смогла восстановить массив b из логов, но чтобы оценить ущерб, нанесённый корпорации, необходимо восстановить похищенный массив a . Это слишком простая задача для Ксюши, поэтому её предстоит выполнить вам.

Входные данные

В первой строке дано целое число t ($1 \leq t \leq 10^4$) — количество наборов входных данных.

Далее следует описание наборов.

В первой строке дано целое число n ($1 \leq n \leq 2 \cdot 10^5$) — количество элементов в массиве a .

Во второй строке даны $2n$ целых числа $b_1, b_2, \ldots, b_{2n}$ ($1 \leq b_i \leq 10^9$) — элементы массива b . Гарантируется, что существует массив a , соответствующий данному массиву b .

Также гарантируется, что сумма n по всем наборам входных данных не превосходит $2 \cdot 10^5$.

Выходные данные

Для каждого набора входных данных выведите n целых чисел — элементы массива a .

входные данные
3
4
1 4 9 5 9 18 1 4
1
10000 10000
2
11021102 999 11022101 999
выходные данные
1 4 4 9
10000
999 11021102

В первом наборе входных данных горилл действовал так:

1. Похитил массив [1, 4, 4, 9];
2. Дописал к массиву его префиксные суммы, получив [1, 4, 4, 9, 1, 5, 9, 18];
3. Перемешал полученный массив, получив [1, 4, 9, 5, 9, 18, 1, 4].